

Build Progress Sheet

AI-Powered Planning-to-Execution Project Progress Intelligence Platform — Backend

Repository lakshgupta260-dev/sih2k26 · Stack FastAPI · PostgreSQL 16 · SQLAlchemy 2 · Alembic · Docker · Updated 01 September 2026

Status of the SIH26122 backend against the eleven-phase plan.

1. Where things stand

2 of 11 phases complete. Both validated against a live PostgreSQL 16 instance rather than assumed working.

Metric	Value
Phases complete	2 / 11 (18%)
Application files	49 Python files, 3,354 lines
Test files	5 files, 1,228 lines, 81 passing tests
Database tables	5 (+ <code>alembic_version</code>)
API endpoints	17 paths
Migrations	1, with zero autogenerate drift
Commits	4 (2 unpushed at time of writing)

2. Phase status

#	Phase	Status	Validated
1	Scaffolding, config, database, Alembic, Docker	Complete	Live PG16 + Docker
2	Auth, RBAC, users, projects	Complete	Live PG16, 81 tests
3	Schedule upload, Excel/CSV parsing, L1-L6, dependencies	Not started	—
4	Report upload, document processing, processing jobs	Not started	—
5	AI extraction, matching, confidence, human review	Not started	—
6	Progress engine, planned-vs-actual analytics, dashboard	Not started	—
7	ML delay prediction, risk scoring, explainability	Not started	—
8	Report generation, notifications	Not started	—
9	Meta / WhatsApp integration	Not started	—
10	Vapi integration, AI project assistant	Not started	—
11	Testing, hardening, performance, documentation	Not started	—

3. What is built

Phase 1 — foundation

Component	Location
Application factory, lifespan, middleware	<code>app/main.py</code>
Typed settings, no hard-coded secrets	<code>app/core/config.py</code>
Domain enumerations	<code>app/core/constants.py</code>
Error hierarchy + single JSON envelope	<code>app/core/exceptions.py</code>
Structured logging, request correlation ids	<code>app/core/logging.py</code> , <code>middleware.py</code>
Declarative base with naming convention	<code>app/db/base.py</code>

UUID / timestamp / soft-delete mixins	<code>app/db/mixins.py</code>
Engine, session lifecycle, DB probe	<code>app/db/session.py</code>
Generic typed repository	<code>app/repositories/base.py</code>
Alembic reading URL from settings	<code>alembic/env.py</code>
Liveness + readiness endpoints	<code>app/api/v1/health.py</code>
Dockerfile, compose (PG16 + Redis 7 + API)	<code>Dockerfile</code> , <code>docker-compose.yml</code>

Phase 2 — auth, RBAC, tenancy

Component	Location
bcrypt hashing, JWT encode/decode	<code>app/core/security.py</code>
Models: user, refresh token	<code>app/models/user.py</code>
Models: project, membership	<code>app/models/project.py</code>
Model: append-only audit log	<code>app/models/audit.py</code>
Auth service (register/login/refresh/logout)	<code>app/services/auth.py</code>
User administration service	<code>app/services/user.py</code>
Project + membership service	<code>app/services/project.py</code>
Audit service	<code>app/services/audit.py</code>
Auth dependencies, role guards, project guards	<code>app/api/deps.py</code>
Routers	<code>app/api/v1/{auth,users,projects}.py</code>
Admin bootstrap CLI	<code>app/cli.py</code>
Migration creating all 5 tables	<code>alembic/versions/20260901_1219_*.py</code>

Tables: `users` , `refresh_tokens` , `projects` , `project_memberships` , `audit_logs` .

Test coverage

File	Tests	Covers
<code>test_config.py</code>	8	Settings contract, DB URL assembly, secret-key enforcement
<code>test_health.py</code>	6	Liveness, readiness, error envelope, OpenAPI
<code>test_auth.py</code>	23	Register, login, tokens, rotation, replay, logout, password change
<code>test_rbac.py</code>	15	Role guards, self-access, role changes, deactivation, admin guards
<code>test_projects.py</code>	24	Project CRUD, visibility scoping, membership, cross-project isolation
Total	76 functions → 81 cases	

4. What is left

Phase 3 — schedule ingestion (next)

- Models: `Schedule` , `Activity` (L1-L6, WBS path, discipline, planned dates, budgeted quantity/UOM), `ActivityDependency` (FS/SS/FF/SF plus lag)
- Excel and CSV parsers with **configurable column mapping** — real schedules never share a column layout
- Primavera P6 (`.xer`) and MS Project (`.xml`) import where feasible
- Hierarchy construction and validation: parent-child integrity, level consistency, dependency cycle detection
- Endpoints: `schedules.py` (upload, list, detail), `activities.py` (list, tree, detail, filter by discipline/level)
- Dependencies to add: `pandas` , `openpyxl`

Phase 4 — document processing and jobs

- Models: `UploadedFile` , `ProcessingJob` (PENDING → PROCESSING → COMPLETED / FAILED), `ProgressReport`
- Abstractions: `DocumentProcessor` , `PDFProcessor` , `ExcelProcessor` , `OCRProcessor` (OCR-ready, provider swappable)

- Celery + Redis workers; upload returns a `job_id` immediately and the frontend polls status
- Upload validation: size cap, extension allow-list, content-type/magic-byte check, per-project storage isolation
- Endpoints: `documents.py`, `reports.py`, job status polling
- Dependencies to add: `pymupdf`, `pdfplumber`, `celery`, `redis`

Phase 5 — extraction and matching (*the core of the problem statement*)

- Extraction layer independent of ORM models (plain dataclasses in `app/ai/`)
- Replaceable `LLMProvider` and `EmbeddingProvider` interfaces
- Matching engine combining: exact activity id, exact code, keyword overlap, fuzzy string similarity, semantic embeddings, discipline agreement, location/chainage agreement, hierarchy proximity
- Configurable confidence thresholds — auto-match / needs-review / unmatched
- Models: `ExtractedActivity`, `ActivityMatch` with confidence and method
- Human review endpoints with complete audit history
- Dependencies to add: `rapidfuzz`, `sentence-transformers`

Phase 6 — progress engine and analytics

- Model: `ActualProgress` — historical, one row per activity per reporting date
- `ProgressService` owning progress computation and roll-up L6 → L1
- Planned-vs-actual variance, S-curve series, completion percentages
- Dashboard aggregation endpoints designed for direct frontend consumption
- Endpoints: `progress.py`, `analytics.py`

Phase 7 — ML delay prediction

- Feature engineering from historical actuals (slip, duration ratio, discipline, dependency depth, weather/idle signals where available)
- Random Forest or XGBoost — explicitly **not** an LLM
- Training pipeline, model persistence and versioning
- Model: `DelayPrediction`, `RiskScore`
- **Data-backed explainability only** — feature contributions from the model, never an LLM inventing plausible-sounding reasons
- Endpoints: `ml.py`, `risks.py`
- Dependencies to add: `scikit-learn`, `joblib`, possibly `xgboost`

Phase 8 — reports and notifications

- Model: `GeneratedReport`; PDF and Excel builders
- Generic `NotificationService` with pluggable channels (in-app, email, WhatsApp) and a `Notification` model
- Endpoints: `notifications.py`, report generation and download
- Dependencies to add: `reportlab` or `weasyprint`

Phase 9 — Meta / WhatsApp

- Webhook verification (`hub.challenge`) and payload signature validation (`X-Hub-Signature-256`)
- Inbound message ingestion into the progress pipeline
- Outbound sending behind a replaceable provider interface
- Endpoint: `integrations/meta.py`

Phase 10 — Vapi and AI assistant

- Vapi webhook handling with signature validation
- Voice-driven progress logging (the PS asks for a low-friction supervisor interface)
- Authenticated AI project assistant that **cannot leak data across projects** — every retrieval scoped by the caller's memberships
- Endpoints: `integrations/vapi.py`, `assistant.py`

Phase 11 — hardening

- Rate limiting on authentication endpoints
- Realistic seed data: multiple phases, Civil / Piping / Electrical / Mechanical / Instrumentation, **100+ L5/L6 activities**, dependencies, planned dates, simulated actuals with delays, daily progress reports

- Production Docker overlay (no `--reload`, gunicorn/uvicorn workers)
- Query performance review and index tuning
- `admin.py` router, coverage sweep, API documentation polish

5. Known gaps and open items

Item	Impact	When
Docker image not rebuilt since Phase 2 deps were added	<code>docker compose up --build</code> needed before the container runs Phase 2 code	Now
No password reset flow	Needs an email provider	Phase 8
No rate limiting on login	Brute-force exposure	Phase 11
Access tokens cannot be revoked before expiry	By design; keep lifetime short	—
Seed data script not written	Spec requires it; needed for demo	Phase 11 (or earlier if useful)
Concurrency on simultaneous refresh untested	Single-threaded test client only	Phase 11
Two commits unpushed	Awaiting explicit approval	Your call

6. Decisions taken (and why)

Decision	Rationale
Synchronous SQLAlchemy, not async	FastAPI runs sync deps in a threadpool; the same session factory is reusable from Celery workers. Heavy work is CPU-bound and lives in workers, so async DB access adds moving parts without buying throughput.
Roles as enum columns, not a <code>roles</code> table	The set is closed and immutable at exactly three values. A lookup table would add a join to every authorization check for no benefit. Flag this if a normalized table is required.
PyJWT over python-jose	Actively maintained; avoids the unmaintained <code>ecdsa</code> dependency.
<code>bcrypt</code> pinned to 4.0.1	passlib 1.7.4 reads <code>bcrypt.__about__</code> , removed in <code>bcrypt</code> ≥ 4.1.
Non-members receive 404, not 403	Confirming a project id exists leaks across the tenancy boundary.
Refresh tokens recorded server-side	A stateless JWT cannot be logged out; without a revocation record, "logout" is a lie.
Audit writes join the caller's transaction	A rolled-back action must not leave a log claiming it happened.
UUID primary keys	Ids appear in URLs and cross-project references; sequential ids are an enumeration risk.

7. Immediate next actions

1. **Rebuild the Docker image** — `docker compose up --build` — so the container picks up the four new Phase 2 dependencies.
2. **Bootstrap an administrator** — `docker compose exec api python -m app.cli create-admin --email you@yourdomain.com`
3. **Push, or keep holding** — 2 commits are staged locally and untouched.
4. **Begin Phase 3** — schedule upload, Excel/CSV parsing, L1-L6 hierarchy, dependencies.

The synthetic labelled dataset built earlier (306-row L1-L6 baseline schedule, 806 labelled field-report lines across four channels, plus ground-truth link labels) is the intended test input for Phases 3 to 6. It is not committed to this repository.